NovaTech Soluções

Especificação de Demanda Técnica

Sistema de Gestão de Pessoas e Salários — Port JavaScript

Empresa	NovaTech Soluções Ltda.
Produto	Sistema de Gestão de Pessoas e Salários
Módulo	Port JavaScript do sistema legado C# (ESIG Group)
Responsável técnico	Coordenadoria de Engenharia de Software
Desenvolvedor designado	Estagiário de Desenvolvimento Full Stack
Versão do documento	1.0
Data de emissão	Março de 2025
Prazo de entrega	A combinar com o responsável técnico

1. Contexto da Demanda

Este documento formaliza a demanda técnica para o port do sistema legado de gestão desenvolvido em C# (ASP.NET WebForms + ADO.NET + SQL Server) para a stack JavaScript moderna aprovada pela coordenadoria de engenharia.

O sistema original — desenvolvido como resposta a um desafio técnico real — gerencia cadastro de pessoas, cargos e cálculo de salários, com geração de relatórios via Crystal Reports e lógica de cálculo encapsulada em stored procedure no banco de dados. A nova versão deve preservar todas as regras de negócio existentes, migrando a arquitetura para Node.js, Express, React e PostgreSQL.

Este documento foi elaborado com o auxílio do Claude (Anthropic), utilizado pelo desenvolvedor como ferramenta de simulação de ambiente profissional. A IA assumiu o papel da coordenadoria técnica: definiu escopo, estruturou o backlog, justificou decisões e estabeleceu critérios de entrega. O código, as decisões técnicas e a execução são inteiramente do desenvolvedor.

2. Stack Aprovada pela Engenharia

A stack abaixo foi definida e aprovada pela coordenadoria técnica. Desvios devem ser justificados e validados antes da implementação.

Camada	Tecnologia	Observação
Back-end	Node.js + Express	*Versão 18+ do Node.js*
Banco de Dados	PostgreSQL + driver pg	*Script DDL obrigatório no repositório*
Front-end	React + Vite + Axios	*Sem frameworks de metaframework (Next.js)*
Testes de API	Postman + Newman	*Collection exportada e versionada no repo*
Testes E2E	Cypress	*Cobertura dos 4 fluxos principais do CRUD*
Gestão de tarefas	GitHub Projects (Kanban)	*Board público e vinculado ao repositório*

3. Requisitos Funcionais

3.1 Entidade: Pessoa

O módulo deve gerenciar a entidade Pessoa com os seguintes campos, preservando a estrutura do sistema legado:

Campo	Tipo	Regra de Negócio
`id`	`SERIAL`	Gerado automaticamente pelo banco
`nome`	`VARCHAR(100)`	Obrigatório
`data_nascimento`	`DATE`	Obrigatório
`cargo_id`	`INTEGER`	Obrigatório — FK para tabela cargo
`email`	`VARCHAR(100)`	Opcional
`telefone`	`VARCHAR(20)`	Opcional
`usuario`	`VARCHAR(50)`	Opcional
`cep`	`VARCHAR(20)`	Opcional
`endereco`	`VARCHAR(200)`	Opcional
`estado`	`VARCHAR(100)`	Opcional
`pais`	`VARCHAR(50)`	Opcional

3.2 Endpoints da API

- **GET /pessoas** — Lista todas as pessoas com suporte a paginação e busca por nome ou ID
- **GET /pessoas/:id** — Retorna uma pessoa pelo ID (404 se não encontrada)
- **POST /pessoas** — Cria uma nova pessoa com validação dos campos obrigatórios
- **PUT /pessoas/:id** — Atualiza os dados de uma pessoa existente
- **DELETE /pessoas/:id** — Remove uma pessoa pelo ID
- **GET /cargos** — Lista todos os cargos disponíveis (necessário para popular o formulário)

- **POST /salarios/calcular** — Dispara o recálculo de salários (equivalente à stored procedure do legado)
- **GET /salarios** — Retorna os salários calculados com suporte a busca por nome ou ID

3.3 Telas do Front-end

- **Gestão de Salários:** tabela com salários calculados, campo de busca e botão para recalcular
- **Cadastro de Pessoas:** tabela com todas as pessoas, campo de busca por nome ou ID
- **Criação/Edição:** formulário com todos os campos da entidade, cargo via select e validação client-side
- **Exclusão:** confirmação antes de deletar o registro

4. Requisitos Técnicos

4.1 Back-end

- Separação clara de responsabilidades: Models, Controllers e Routers em diretórios distintos
- Middleware dedicado para validação do body das requisições POST e PUT
- Middleware global de tratamento de erros com respostas padronizadas
- Credenciais de banco de dados exclusivamente via variáveis de ambiente (.env)
- Arquivo .env.example versionado com todas as variáveis necessárias

4.2 Testes de API (Postman)

- Collection completa com todos os endpoints documentados e com exemplos de body
- Environment com variável base_url e demais variáveis necessárias
- Testes automatizados em cada requisição: status code, schema e campos esperados
- Cobertura de cenários de erro: 404, 400 e campos inválidos
- Arquivos .json da Collection e Environment versionados na pasta /postman

4.3 Testes E2E (Cypress)

- Cobertura dos quatro fluxos principais: listar, criar, editar e excluir
- Teste de validação de formulário com campos obrigatórios vazios
- Teste de feedback visual: mensagens de sucesso e erro
- Comando para rodar os testes documentado no README

4.4 Gestão de Tarefas

- Board Kanban público vinculado ao repositório GitHub
- Issues criadas por camada técnica com título descritivo e critérios de aceite

- Uso de labels para categorizar as issues por área

5. Critérios de Aceite da Entrega

A entrega será considerada completa quando todos os itens abaixo estiverem atendidos:

Critério	Peso	Observação
Qualidade e organização do código	**25%**	*Estrutura, nomenclatura, separação de responsabilidades*
Cobertura de testes (Postman + Cypress)	**30%**	*Cenários cobertos, assertions, cenários de erro*
CRUD funcionando de ponta a ponta	**20%**	*Todos os endpoints e telas operacionais*
Documentação (README + Postman)	**15%**	*Clareza, completude e reprodutibilidade*
Gestão com GitHub Projects	**10%**	*Issues bem descritas, board organizado*

> **Nota da coordenadoria**
> Uma entrega 70% completa, bem documentada e com testes parciais tem mais valor do que uma entrega 100% funcional sem documentação nem testes. O objetivo é construir o hábito de trabalhar com qualidade desde o início da carreira.

6. Diferenciais Valorizados

Os itens abaixo não são obrigatórios para a entrega, mas serão considerados positivamente pela coordenadoria na avaliação do trabalho:

- Deploy da aplicação em plataforma gratuita (Railway, Render, Vercel + Neon)
- Histórico de commits limpo com mensagens no padrão Conventional Commits
- Implementação de busca e filtros avançados na listagem
- Separação de variáveis de ambiente para desenvolvimento e produção
- Tratamento de erros detalhado no front-end com mensagens amigáveis ao usuário

7. Entregáveis Esperados

7.1 O repositório deve conter obrigatoriamente:

- **README.md** com descrição, pré-requisitos, instruções de instalação e como executar os testes

- **Pasta /postman** com os arquivos .json da Collection e do Environment exportados
- **.env.example** com todas as variáveis de ambiente necessárias (sem valores reais)
- **Script DDL (.sql)** para criação do banco de dados e das tabelas
- **Board público no GitHub Projects** com as issues organizadas por camada técnica

7.2 Checklist de entrega:

1. Repositório público e acessível no GitHub
2. Board do GitHub Projects visível via link público
3. README com instruções claras de como rodar o projeto localmente
4. Collection do Postman executável via Newman sem erros
5. Testes Cypress rodando sem falhas no ambiente local

8. Nota Sobre o Uso de Inteligência Artificial

Este documento foi elaborado com o auxílio do Claude (Anthropic), utilizado pelo desenvolvedor como ferramenta de simulação de ambiente profissional. A IA assumiu o papel da coordenadoria técnica: definiu escopo, estruturou o backlog, justificou decisões e estabeleceu critérios de entrega.

O que a IA fez	O que a IA não fez
• Simulou o papel de coordenador/gestor técnico • Gerou este documento de especificação • Estruturou o backlog por camada técnica • Debateu e justificou decisões (ex: PostgreSQL) • Estabeleceu critérios de entrega e qualidade	• Escrever qualquer linha de código do projeto • Tomar decisões técnicas pelo desenvolvedor • Resolver problemas de implementação • Executar ou validar os testes • Substituir o aprendizado técnico

Saber utilizar IA como ferramenta de processo — e não como substituto do desenvolvimento — é, em si, uma competência relevante no mercado de trabalho atual.

9. Avisos

Atenção

• Entregas sem testes (Postman ou Cypress) não serão consideradas completas pela coordenadoria.

• Credenciais reais (senhas, tokens, chaves de API) não devem constar em nenhum arquivo do repositório.

• Dúvidas técnicas sobre a especificação devem ser levadas ao responsável técnico antes de iniciar a implementação.

— Coordenadoria de Engenharia de Software, NovaTech Soluções